

SQL - Trigger e Procedure

Disciplina: Banco de Dados

Prof. Tiago Piperno Bonetti

bonetti@prof.unipar.br

Triggers (gatilhos)

Em muitos casos é conveniente especificar um tipo de **ação** a ser tomada quando certos **eventos ocorrem**.

Os Triggers são **gatilhos** que disparam a execução de **códigos armazenados** no servidor, ou outros códigos SQL, **sem** a necessidade de uma **chamada específica**.

A principal finalidade de uma trigger é permitir que você **automatize** certas **ações** ou aplique **regras** de **negócio** específicas no banco de dados, garantindo a consistência e a integridade dos dados.

Por exemplo, você pode usar uma trigger para executar uma determinada ação quando um **novo registro** é **inserido** em uma tabela, como **atualizar valores** em outras tabelas relacionadas ou registrar informações adicionais.

Triggers (gatilhos)

Eventos que podem acionar um Trigger.

- **INSERT**
- **UPDATE**
- **DELETE**

Triggers (gatilhos)

Mecanismos úteis para alertar usuários ou iniciar tarefas automaticamente quando certas condições forem satisfeitas.

Exemplos:

- Atualizar estoque em sistemas de venda.
- Inserir empréstimos ao utilizar limite em sistemas bancários.
- Incrementar contador.
- Alertar estoque mínimos.

Triggers (gatilhos)

Trigger será disparado:

- **BEFORE** = Antes do evento
- **AFTER** = Depois do evento

Modo	Descrição
BEFORE INSERT	Disparado antes de uma ação de inserção.
BEFORE UPDATE	Disparado antes de uma ação de alteração.
BEFORE DELETE	Disparado antes de uma ação de exclusão.
AFTER INSERT	Disparado depois de uma ação de inserção
AFTER UPDATE	Disparado depois de uma ação de alteração.
AFTER DELETE	Disparado depois de uma ação de exclusão.

Variáveis de Contexto

Acessando os registros:

- **NEW.nome_coluna** – Quando estamos inserindo uma nova linha.
- **OLD.nome_coluna** – Quando estamos excluindo uma linha.

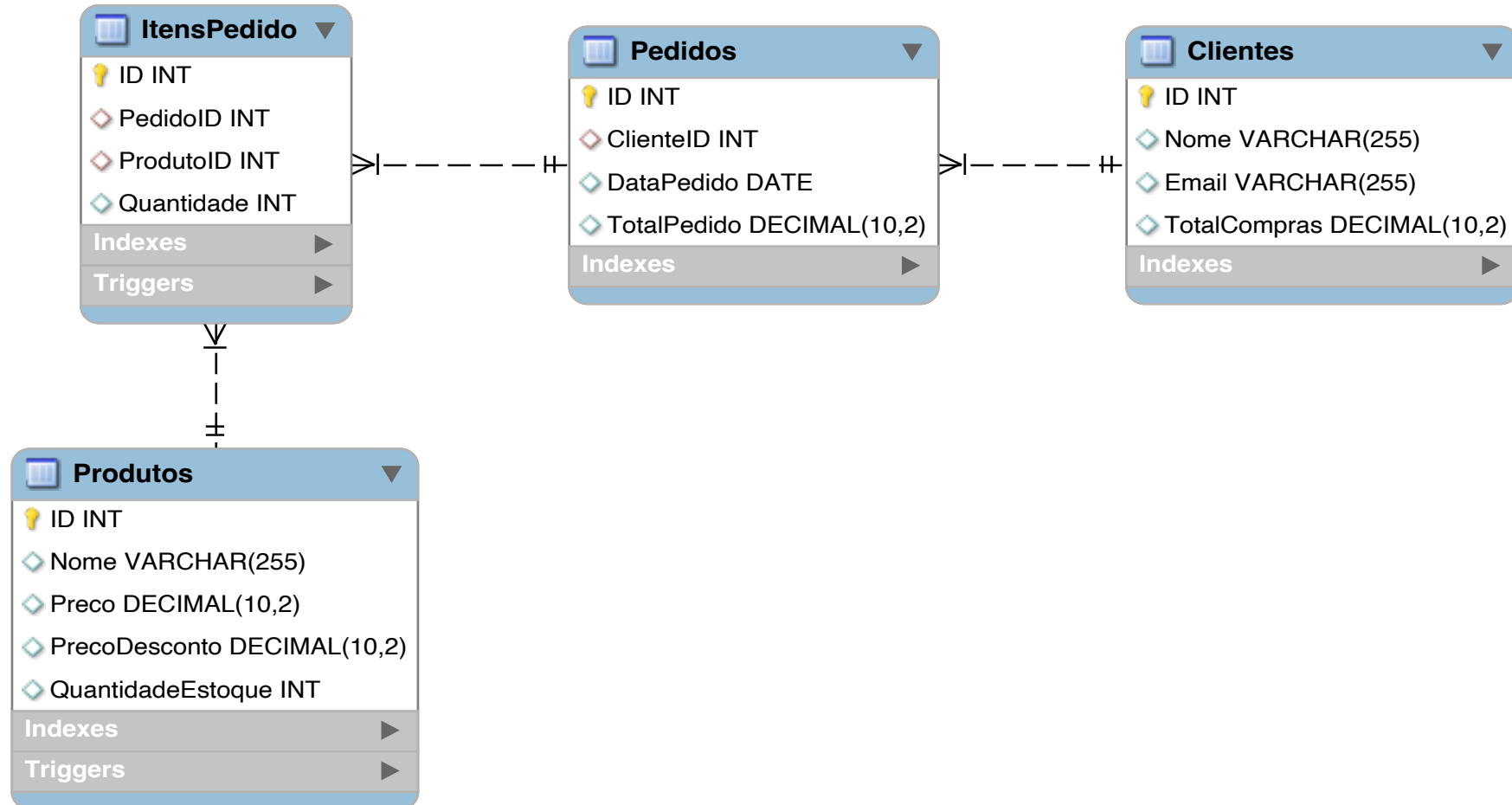
Evento	Quando é disparado	Variáveis
Before insert	Antes de uma inserção	New
After insert	Depois de uma inserção	New
Before update	Antes de uma atualização	New e Old
After update	Depois de uma atualização	New e Old
Before delete	Antes de uma exclusão	Old
After delete	Após uma exclusão	Old

Triggers (gatilhos)

Sintaxe

```
CREATE TRIGGER Nome  
{BEFORE / AFTER} {INSERT / UPDATE / DELETE}  
ON Tabela  
FOR EACH ROW  
BEGIN  
    . . .  
END;
```

Triggers (gatilhos) - Exemplos



Triggers (gatilhos) - Exemplos

Imagine que uma loja tem por praxe dar 10% de desconto para pagamentos a vista. Antes de inserir um novo produto no banco de dados calcule o valor do produto com desconto e insira esse valor no atributo “Preco_Desconto.

```
DELIMITER //
```

```
CREATE TRIGGER Desconto_Produto  
BEFORE INSERT ON Produtos  
FOR EACH ROW  
BEGIN  
    SET NEW.PrecoDesconto = (NEW.Preco * 0.90);  
END //
```

```
DELIMITER ;
```

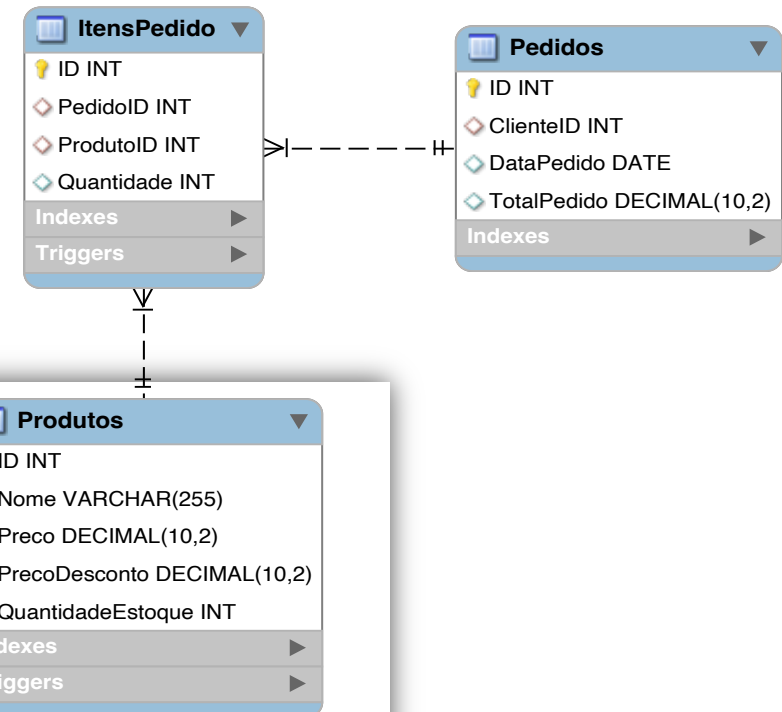


Triggers (gatilhos) - Exemplos

Crie uma trigger para que cada vez que um registro aconteça na tabela itensPedido o campo TotalPedido da tabela Pedido seja atualizado somando o valor atual mais o novo item inserido.

```
CREATE TRIGGER atualizar_total_pedido
AFTER INSERT ON ItensPedido
FOR EACH ROW
BEGIN
    DECLARE valor_item DECIMAL(10, 2);
    SET valor_item = (SELECT Preco
                      FROM Produtos WHERE ID = NEW.ProdutoID);

    UPDATE Pedidos
    SET TotalPedido = TotalPedido + (valor_item * NEW.Quantidade)
    WHERE ID = NEW.PedidoID;
END //
```



Stored Procedures (Procedimentos Armazenados)

Módulos de programas armazenados pelo SGBD no servidor de banco de dados, **ativados** por aplicações, triggers ou outras rotinas.

Um **conjunto** de comandos **SQL** que podem ser **armazenados** no servidor.

Quando usar?

Quando **várias aplicações** clientes são escritas em diferentes linguagens ou funcionam em diferentes plataformas, mas precisam realizar as **mesmas operações** de banco de dados.

Quando a **segurança é prioridade**. Bancos, por exemplo, usam Stored Procedures para todas as operações comuns. Isto fornece um ambiente consistente e seguro.

Stored Procedures - Vantagens

- ✓ **Centralização:** Garante a correta utilização do banco independente de sistemas.
- ✓ **Segurança:** Aplicações e usuários não possuem nenhum tipo de acesso as tabelas do banco de dados diretamente.
- ✓ **Performance:** Menos informações precisam ser enviadas entre o servidor e o cliente. (porem aumenta a carga no sistema do servidor de banco de dados).

Stored Procedures - Sintaxe

Criando uma Stored Procedure:

```
DELIMITER //  
    CREATE PROCEDURE    Nome (Parâmetros)  
        BEGIN  
  
        END //  
DELIMITER ;
```

Stored Procedures - Parâmetros

Parâmetros são opcionais, porém tornam as Stored Procedures mais úteis.

Sintaxe básica:

(<MOD0> <Nome> <TIPO>)

Stored Procedures - Parâmetros

Modo:

- ✓ **IN:** Indica que um parâmetro pode ser passado, mas qualquer alteração dentro do Stored Procedure não altera o parâmetro.
- ✓ **OUT:** Indica que o Stored Procedure pode alterar o parâmetro e devolve-lo ao programa de chamada.
- ✓ **INOUT:** É a combinação do modo IN e OUT, você pode passar parâmetros para o Stored Procedure e recuperá-los com um novo valor a partir do programa de chamada.

Stored Procedures - Executando

CALL *NomeStoredProcedure* (**Parâmetro**)

Ou

SET *@Nome_Variavel* = 'Valor';

CALL *NomeStoredProcedure* (*@Nome_Variavel*);

Stored Procedures - Exemplos

✓ Criando uma Stored Procedure que realiza uma consulta na tabela Produtos.

```
-- Criando a procedure
Delimiter //
CREATE PROCEDURE ConsultaProd ()
    BEGIN
        SELECT * FROM Produtos;
    END //
Delimiter ;

-- Executando a procedure
CALL ConsultaProd;
```

Stored Procedures - Exemplos

```
-- Criando a procedure
Delimiter //
CREATE PROCEDURE ConsultaProdParametro
(IN NomeConsulta VARCHAR(45))
BEGIN
    SELECT * FROM Produtos
    WHERE Nome = NomeConsulta;
END //
Delimiter ;

-- Executando a procedure
CALL ConsultaProdParametro('Camiseta');
-- Utilizando variável
SET @Nome_Prod = 'Camiseta';
CALL ConsultaProdParametro(@Nome_Prod);
```

✓ Criando uma Stored Procedure que realiza uma consulta na tabela Produtos passando como parâmetro o nome do produto.

Stored Procedures - Exemplos

```
Delimiter //
```

```
CREATE PROCEDURE BuscaNome  
(IN IDConsulta INT, OUT NomeReposta VARCHAR (50))  
  BEGIN  
    SELECT Nome  
    INTO NomeReposta  
    FROM Produtos  
    WHERE ID = IDConsulta;  
  END //
```

```
Delimiter ;
```



```
CALL BuscaNome (1, @NomeProduto);  
SELECT @NomeProduto;
```

✓ Criando uma Stored Procedure que realiza uma consulta na tabela Produtos passando como parâmetro o ID do produto retornando o Nome.

Stored Procedures - Exemplos

- ✓ Criando uma Stored Procedure que realiza a subtração do valor de desconto ao valor do produto.

```
Delimiter //
```

```
CREATE PROCEDURE CalculaDesconto
```

```
(INOUT Valor Decimal (10,2), IN Desconto DECIMAL (10,2))
```

```
    BEGIN
```

```
        SET Valor = Valor - Desconto;
```

```
    END //
```

```
Delimiter ;
```



```
SET @ValorProduto = 800.00;
```

```
CALL CalculaDesconto(@ValorProduto, 200.00);
```

```
SELECT @ValorProduto;
```